Ders 17 Çalışma Özeti

Ders 17 Çalışma Özeti

Genel Konular

- Bu kayıt uygulama oturumudur.
  - Asıl anlatım yaklaşık 00:05:15'te başlar. Kayıt yaklaşık 00:42:00'de biter.
- Arayüzler (interface)
  - Tanımı, metot imzaları, sabitler, `default` ve `static` metotlar.
  - `implements` ile birden fazla arayüz uygulama.
  - Örnek: Birbiriyle ilgisiz `Book` ve `Rectangle` sınıflarının ortak `Comparable` arayüzünü uygulaması.
- İstisnalar (exception)
  - `try`, `catch`, `finally`, `throw`.
  - Sıfıra bölme ve dizi sınırı dışına çıkma örnekleri.
- Jenerikler (generics)
  - Jenerik metot ve jenerik sınıf.
- Koleksiyonlar (collections)
  - `ArrayList`, `LinkedList`, `Iterator`, `HashMap`.
  - Örnek: `ArrayList`'ten kalıtım alan, negatif indeksi destekleyen ve silinen elemanları kaydeden `MyArrayList<E>` sınıfı.

Hocanın Özellikle Vurguladığı Kısımlar

- Arayüz iki sebeple kullanılır.
  - Java'da `extends` tek sınıfa yapılır. `implements` ile birden fazla arayüz uygulanır. Böylece çoklu kalıtıma benzer bir yapı kurulur.
  - Daha iyi tasarım için. Sınıfın içine bakmadan, yalnızca hangi arayüzü uyguladığına bakarak hangi metotları çağırabileceğimizi biliriz.
- Arayüzü uygulayan sınıf, arayüzdeki bütün soyut metotları yazmak zorundadır.
- Hata çıkabilecek kodu `try` bloğuna yazın. Hata türüne göre ne yapılacağını `catch` bloklarına yazın.
  - `try` içinde hata çıkınca o satırın altındaki satırlar çalışmaz.
- Jenerik yapı, aynı işi her tip için ayrı ayrı yazmayı önler.
- Veri yapıları da nesneye yönelik tasarlanmıştır. İç yapısını bilmeden `extends` ile mevcut bir veri yapısının davranışını değiştirebilirsiniz.
- `HashMap` çok kullanışlı bir veri yapısıdır. Anahtarla ortalama $O(1)$ sürede arama yapar.

Kısa Tekrar Notları

- `interface` tanımı `class` tanımına benzer. `class` yerine `interface` yazılır.
- Arayüzdeki metotlar gövdesizdir. `abstract` yazılmasa da soyut sayılır.
- Arayüzdeki değişkenler `public static final` olur. `final` yazılmasa da değiştirilemez.
- `default` metodun gövdesi vardır. Uygulayan sınıf onu yazmak zorunda değildir.
- Arayüzler birbirinden `extends` ile kalıtım alabilir.
- `catch` sırası: önce özel, sonra genel istisna.
- `finally` bloğu istisna olsa da olmasa da çalışır.
- Jenerik metot: `public static <T> void yazdir(List<T> liste)`.
- `ArrayList`: boyutu dinamik dizi. `LinkedList`: düğümler bellekte ardışık olmak zorunda değil.

- Bütün `Collection` sınıfları `add`, `remove`, `size` gibi aynı metotları paylaşır.
- `HashMap` anahtar-değer (key-value) çifti tutar. `put`, `get`, `remove`, `clear` metotları vardır.

Detaylı Açıklamalar

Arayüz örneği: Comparable

Hoca, birbiriyle ilgisi olmayan iki sınıfa ortak bir "büyüklük-küçüklük" arayüzü uyguladı. Kitapta ölçüt fiyattır, dörtgende alandır. Soyutlama aynıdır.

```
interface Comparable {
    public int isBiggerThan(Comparable c);
}

class Book implements Comparable {
    private String name;
    private double price;

    public Book(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() { return price; }

    @Override
    public int isBiggerThan(Comparable c) {
        Book other = (Book) c;
        if (getPrice() > other.getPrice()) return 1;
        else if (getPrice() == other.getPrice()) return 0;
        else return -1;
    }
}
```

- `Rectangle` sınıfı aynı metodu alanına (`x * y`) göre yazar.
- Renk gibi büyüklük ilişkisi kurulamayan sınıflar bu arayüzü uygulamaz.

İstisna akışı

- `try` bloğunda bir satır istisna fırlatınca kalan satırlar atlanır.
- Uygun türdeki `catch` bloğu çalışır. Sonra program `try-catch` yapısından sonra devam eder.
- `finally` bloğu her durumda çalışır. Dosya kapatma gibi temizlik işleri için uygundur.
- `throw new Exception("...")` ile kendimiz istisna fırlatabiliriz.

```
try {
    int b = a / 0;          // ArithmeticException
    int[] d = new int[3];
    d[5] = 1;               // ArrayIndexOutOfBoundsException
} catch (ArithmeticException e) {
    System.out.println("Sıfıra bölme");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Dizi sınırı aşıldı");
} finally {
    System.out.println("Her durumda çalışır");
}
```

Jenerikler

- Bir listenin elemanlarını yazdıran metot, elemanın tipinden bağımsızdır.
- Metot jenerik yazılınca `Integer`, `Double` ve `Character` listeleriyle aynı kod çalışır.

- Sınıflar da jenerik olabilir. Örneğin fiyat alanı bazen `Double`, bazen `Integer` olabilir.

Koleksiyonlar

- `List` bir arayüzdür. `ArrayList` ve `LinkedList` bu arayüzü uygular.
- `ArrayList` ile elemanlara indeksle doğrudan erişilir.
- `LinkedList` ile düğümler sırayla gezilir. Hoca, editördeki "geri al / ileri al" durumlarını örnek verdi.
- `Iterator` ile `hasNext()` / `next()` döngüsü kurulur.
- `retainAll` iki koleksiyonun kesişimini tutar.
- `HashMap` örneği: `put(17, yeniAd)` aynı anahtardaki eski değerin (`eskiAd`) üzerine yazar. `clear()` sonrası `get` sonucu `null` olur.

MyArrayList: veri yapısını kalıtımla genişletmek

Python'da `liste[-1]` son elemanı verir. Java'da `ArrayList.get(-1)` istisna fırlatır. Hoca, `get` metodunu override ederek negatif indeksi destekledi. Sonra `remove` metodunu da override ederek silinen elemanları ayrı bir listede tuttu.

```
class MyArrayList<E> extends ArrayList<E> {
    private ArrayList<E> removedItems = new ArrayList<>();

    @Override
    public E get(int index) {
        if (index < 0)
            index = super.size() + index;
        return super.get(index);
    }

    @Override
    public E remove(int index) {
        E silinen = super.remove(index);
        removedItems.add(silinen);
        return silinen;
    }

    public ArrayList<E> getRemovedItems() { return removedItems; }
}
```

- Derste ilk deneme çalışmadı. `if` içinde `super.get(super.size() + index)` sonucu döndürülmüyordu. Bir öğrenci `index = super.size() + index;` yazmayı önerdi. Bu düzeltmeyle `get(-1)` son elemanı (81) verdi.
- Test kodu ve çıktısı (Java ile doğrulandı):

```
MyArrayList<Integer> listem = new MyArrayList<>();
for (int i = 0; i < 10; i++) listem.add(i * i);
System.out.println(listem.get(-1));   // 81
listem.remove(3); listem.remove(5);
for (int i = 0; i < 10; i++) listem.add(i * i);
listem.remove(11); listem.remove(12);
System.out.println(listem);
// [0, 1, 4, 16, 25, 49, 64, 81, 0, 1, 4, 16, 36, 49, 64, 81]
System.out.println(listem.getRemovedItems());   // [9, 36, 9, 25]
```

- Hocanın mesajı: `ArrayList`'in içini bilmeden, yalnızca bir metodu override ederek kendi ihtiyacımıza göre veri yapısı oluşturabiliriz. İstersek `Collection` arayüzünü doğrudan uygulayıp sıfırdan bir veri yapısı da yazabiliriz.

Ders sonrası notlar

- Derste yazılan `isBiggerThan` metodunda ikinci koşul da `>` olarak kalmıştı (`else if (getPrice() > other.getPrice()) return 0;`). Bu yüzden eşit değerlerde beklenen 0 dönmedi.

Doğrusu yukarıdaki gibi == kontrolü ve son durumda `-1` döndürmektir.

- Java'nın kendi `java.lang.Comparable<T>` arayüzü vardır. Metodu `compareTo(T o)`'dur. Derste aynı adla kendi arayüzü yazıldı. Gerçek projede `Comparable<Book>` uygulamak daha doğrudur. O zaman `Collections.sort` gibi hazır metotlar da çalışır.
- Hoca "jenerik kodun hangi tiple çalışacağı runtime'da belirlenir" dedi. Doğru değil. Jenerik tip kontrolü derleme zamanında yapılır. Çalışma zamanında tip bilgisi silinir (type erasure).
- `Iterator` yalnızca ileri gider. "İki ileri, bir geri" gibi gezinme için `ListIterator` ve onun `previous()` metodu kullanılır.
- `HashMap` aramasının $O(1)$ olması ortalama durum içindir. Çok çakışma (collision) olursa yavaşlar.
- Hoca `HashMap` için de "hepsi bir `Collection`" dedi. `HashMap`, `Collection` hiyerarşisinde değildir. `Map` arayüzünü uygular.
- Hoca bir yerde "`finally` olmadığı için program burada bitiyor" dedi, sonra akışın devam ettiğini söyledi. Doğrusu ikincisidir: Uygun `catch` çalıştıktan sonra program `try-catch` yapısının altından devam eder (Java ile denendi).
- `ArrayList.get(-1)`: Hoca derste `ArrayIndexOutOfBoundsException` gösterdi. JDK 25 ile `IndexOutOfBoundsException: Index -1 out of bounds for length 1` alındı (Java ile denendi). Fark büyük olasılıkla JDK sürümündendir; `ArrayIndexOutOfBoundsException` da `IndexOutOfBoundsException`'ın alt sınıfıdır.
- `MyArrayList<Integer>` üzerinde `remove(3)` çağrısı 3 değerini değil, 3. indeksteki elemanı siler. `Integer` listesinde değere göre silmek için `remove(Integer.valueOf(3))` yazılır.
- Kayıtta 00:11:12 civarı ("Ich sehe so, Jack") ve 00:38:35 civarı ("And it's just public") anlamsız transkripsiyon parçalarıdır. Bu anlarda asistan kod yazıyor.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.